

MSE 312: MECHATRONICS DESIGN II

Integration Lab 4: Motor Control Experiments

OBJECTIVES

- To become familiar with Matlab/Simulink tools for motor control

Required Components/Material:

- Matlab/Simulink 2021b
- MEGA 2560 R3 Board ATmega2560 ATMEGA16U2 + USB Cable for Arduino
- H-bridge circuit, GM8224S009 DC Servo Gearmotor, Arduino Jumper Cables

References:

- Arduino Support from MATLAB/Simulink (www.mathworks.com)
- Rensselaer Arduino Support Package Library (RASPLib)
<https://www.mathworks.com/matlabcentral/fileexchange/62702-reusselaer-arduino-support-package-library-rasplib>

PROCEDURE

ACTIVITY 4.1: DC Motor open loop response

Let us first examine the open loop response of the dc motor and compare it with the expected theoretical results.

Example 4.1: Speed and Position Open Loop Response

1. Combine encoder reading from lab 2 (example 2.2) and motor drive from Activity 3.3 (lab 3) in a subsystem to create a model for the dc motor with voltage as the input and angular position and velocity of the shaft as the output as shown in Fig 4.1.



Figure 4.1: Subsystem created for DC motor.

To obtain the angular speed from the shaft position, use the “filtered derivative” block from *Simulink* > > *continuous* and set the Coefficient *cc* in the transfer function approximation $s/(cs + 1)$ used for linearization to 0.01. Smaller values of *c* result in noisier responses.

2. Now use a gain with a slider, three “manual switch” blocks, and a “scope” with two inputs to test the motor position and speed in response to the following inputs:

- A “Sine wave” with amplitude 1, frequency 1 rad/s, and sampling time 0.01.
- A constant
- A pulse wave with amplitude 1, period 2s, and a pulse width of 50%
- A single pulse with amplitude 1 (between 1 and 2 seconds) created with a “signal builder” block from *Simulink>>sources*

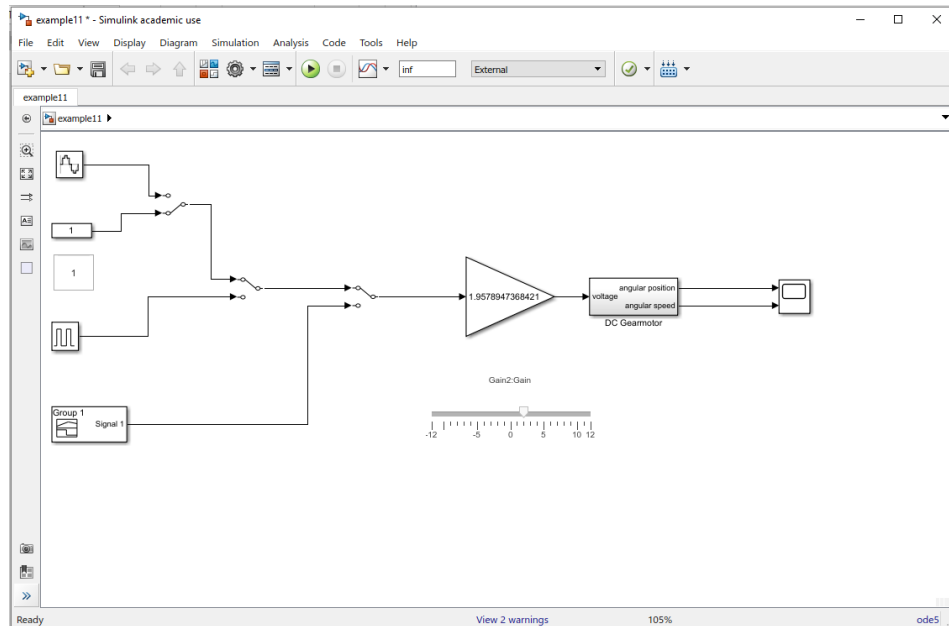


Figure 4.3: Open-loop DC motor test with different inputs.

- Run the simulation and inspect the system response with different input types and gain values. Note that these gain values are indicative of input amplitude.

ACTIVITY 4.2: Position Control of a DC motor

Example 4.2: Closing the loop with a PI(D) controller

Create a closed loop model of the system using the following components

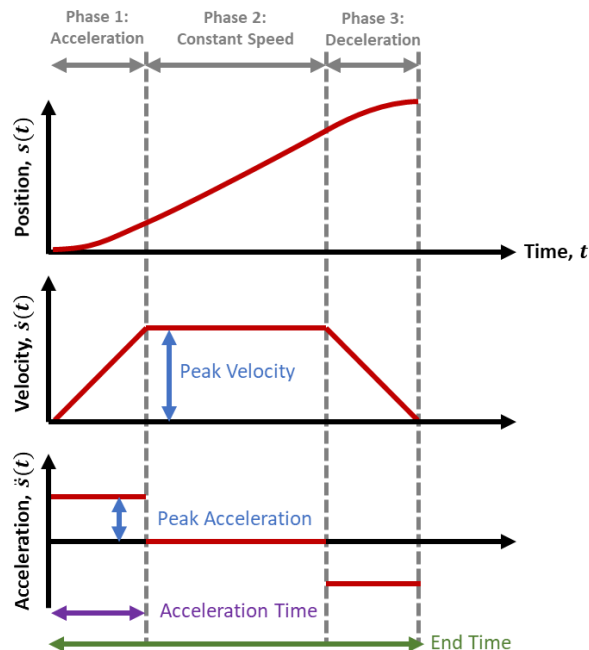
- The “dc gearmotor” subsystem you created in the previous example (as the system model).
- A constant block to set the reference input (desired position) in radians.
- A sum block to calculate the error between the set-point and the current position.
- A “PID controller” block from *Simulink>>continuous*. Connect three separate sliders and “edit” blocks to the P, I and D components of the PID controller respectively to be able to change these control coefficients while the simulation is running.
- Multiple “scope” blocks to see the set-point, error, control signal, and angular position and velocity of the motor.

Exercise: PID Controller Tuning

1. Set I and D coefficients to zero and choose a small P for start (for example $p=1$).
2. To examine the closed-loop response with this proportional gain P, apply set-point changes in the input and observe the results. Begin with small set-points. Increase the set-point from -3 to 3 rad once in one big step and once gradually in smaller steps. What happens when sudden changes in reference values are used? Compare and explain the results. What solutions would you suggest to avoid sudden input changes?
3. Change the P gain manually and study the response to achieve the desired behavior.
4. Try different values of P, I and D to see how the effect on the response of your system and justify the results.

Exercise: Speed/position Control using a trapezoidal speed trajectory

Use a trapezoidal speed trajectory to move the motor from point A (at zero speed) to point B (at zero speed) in T seconds (see course material for details). Consider the effect of changing your controller parameters (P, I and D) on the performance of the system.



Taken from: <https://www.mathworks.com/help/robotics/ug/design-a-trajectory-with-velocity-limits-using-a-trapezoidal-velocity-profile.html>

Model configuration instructions
(detailed images in example 1.3 (Integration Lab 1))

1. From the dropdown menu by the gear icon on top of the Simulink model, choose the model configuration parameters option.
2. From the configuration parameters window that opens, choose hardware implementation on the left-hand side menu and select the Arduino Mega 2560 as the hardware board
3. After choosing the Mega 2560 as your Hardware board, the Target hardware resources option appears at the bottom of the window. From Groups, select Host-board connection and choose the set host COM port as manually and insert the COM port number your Arduino is connected to (e.g., COM 3). Click OK to apply the changes.
4. In the the Solver tab, for the Start time field, enter 0. In the Stop time field, enter the amount of time you want your model to run. For example, enter inf if you want to run the program indefinitely. From the Type list, choose Fixed-step. From the Solver list, choose a solver. For example, ode5 (Dormand-Prince). In the Fixed step size field, enter a sample time. For example, enter 0.01 seconds for a sample rate of 100 samples/second. This sample time should be the same as the sample time in your blocks. It is recommended to keep the sample time constant in all blocks.
5. In the simulink model window, choose the simulation mode as External. Save the model and you are ready to run the simulation.